

Regressão Logística Binária

Tutorial Completo — Teoria, Formalismo e Implementação em R e Python

O tutorial nasceu de notas de estudos feitas manuscritas. As leituras que embasaram este estudo são de livros de estatística e ciência de dados que serão referenciadas nesta apresentação.

O manuscrito que deu origem a esta apresentação foi Manuscrito Original

Caso esta apresentação não este renderizando de forma adequada use o pdf gerado ou o html para realizar a leitura!

Autor: Paulo Pimenta

Sobre este tutorial

Cobertura completa da regressão logística binária: da motivação matemática até a implementação prática em **R** e **Python**. O código aceita qualquer arquivo **.csv** — basta ajustar o bloco de configuração indicado em cada seção.

Baseado em notas de aula revisadas, com formalismo estatístico e conteúdos complementares ausentes no manuscrito original.

1. Introdução e Motivação

1.1 Por que não usar regressão linear?

Quando a variável resposta y é **binária** — assume apenas os valores 0 (fracasso, ausência, não-evento) e 1 (sucesso, presença, evento) — a regressão linear ordinária apresenta problemas fundamentais:

Problema	Descrição
Previsões fora de $[0,1]$	$\hat{y} = \mathbf{x}^\top \boldsymbol{\beta}$ pode assumir qualquer valor real, resultando em “probabilidades” negativas ou maiores que 1
Heterocedasticidade estrutural	A variância de $y \sim \text{Bernoulli}(p)$ é $p(1-p)$, que varia com p , violando a homocedasticidade
Distribuição dos resíduos	Os resíduos não seguem distribuição normal, invalidando os testes da regressão linear

A solução é modelar diretamente a **probabilidade condicional** $P(Y = 1 \mid \mathbf{x})$ por meio de uma função que mapeie $(-\infty, +\infty) \rightarrow (0, 1)$. A função escolhida é a **sigmoide logística**.

1.2 Aplicações típicas

- **Medicina:** diagnóstico (doente/saudável), presença de doença (sim/não)
- **Finanças:** inadimplência (sim/não), fraude (sim/não)

- **Marketing:** conversão de cliente (compra/não compra), *churn* (cancela/permanece)
- **NLP:** detecção de spam (spam/não-spam), sentimento (positivo/negativo)
- **Biologia:** sobrevivência de espécie (sobrevive/extingue)

1.3 Fluxo da análise

O pipeline completo de uma análise de regressão logística binária:

```
Dados .csv
|
v
1) Exploração
|
v
2) Estimção do modelo
|
v
3) Avaliação do modelo
|
v
4) Testes de hipóteses
|
v
5) Previsão
```

2. O Modelo Logístico

2.1 A função sigmoide

A equação central do modelo é:

$$\hat{y} = P(Y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-z}}, \quad z = a_1x_1 + a_2x_2 + \dots + a_px_p + b$$

onde:

Símbolo	Nome	Descrição
$x_1, \dots, x_p$	Variáveis preditoras	Características observadas
$a_1, \dots, a_p$	Coefficientes de regressão	Parâmetros estimados pelo modelo
b	Intercepto	Parâmetro constante (viés)
z	Log-odds (logit)	Combinação linear das preditoras
$\hat{y}$	Probabilidade predita	$P(Y = 1 \mathbf{x}) \in (0, 1)$

Propriedade fundamental: independentemente do valor de z , $0 < \hat{y} < 1$ sempre. O modelo nunca produz uma probabilidade inválida.

```

library(ggplot2)
library(dplyr)
library(pROC)
library(caret)
library(gridExtra)

z_vals <- seq(-10, 10, length.out = 600)
sig_df <- data.frame(z = z_vals, y = 1 / (1 + exp(-z_vals)))

ggplot(sig_df, aes(x = z, y = y)) +
  geom_ribbon(data = filter(sig_df, y >= 0.5),
            aes(ymin = 0.5, ymax = y), fill = "#27AE60", alpha = 0.18) +
  geom_ribbon(data = filter(sig_df, y < 0.5),
            aes(ymin = y, ymax = 0.5), fill = "#C0392B", alpha = 0.18) +
  geom_line(color = "#2C3E50", size = 1.4) +
  geom_hline(yintercept = c(0, 0.5, 1), linetype = "dashed",
            color = c("gray70", "gray40", "gray70"), size = 0.6) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray40", size = 0.6) +
  geom_point(data = data.frame(z = 0, y = 0.5),
            aes(x = z, y = y), color = "#E74C3C", size = 3) +
  annotate("text", x = 6.5, y = 0.18,
            label = "hat(y) == frac(1, 1 + e^{-z})",
            parse = TRUE, size = 4.5, color = "#2C3E50") +
  annotate("text", x = 4, y = 0.80, label = "Classe 1", color = "#27AE60",
            fontface = "bold", size = 4) +
  annotate("text", x = -4, y = 0.18, label = "Classe 0", color = "#C0392B",
            fontface = "bold", size = 4) +
  scale_y_continuous(breaks = c(0, 0.25, 0.5, 0.75, 1),
                    labels = c("0", "0,25", "0,5", "0,75", "1")) +
  scale_x_continuous(breaks = seq(-10, 10, 2)) +
  labs(x = "z (log-odds)", y = expression(hat(y) == P(Y == 1 ~ "|" ~ bold(x))),
       title = "Função Logística (Sigmoid)") +
  theme_minimal(base_size = 13) +
  theme(panel.grid.minor = element_blank())

```

2.2 A transformação logit e os odds

Invertendo a função sigmoide:

$$\log_e \left(\frac{p}{1-p} \right) [\text{logit}(p)] = z = a_1 x_1 + \dots + a_p x_p + b$$

O termo $\frac{p}{1-p}$ é a **razão de chances** (*odds*): razão entre a probabilidade de ocorrência e de não-ocorrência do evento. A transformação logit lineariza a relação entre as preditoras e os odds do evento, que é a base para interpretar os coeficientes.

Interpretação dos coeficientes via Odds Ratio

$$OR_i = e^{a_i}$$

Situação	Interpretação
$a_i > 0 \Rightarrow OR_i > 1$	Aumento de 1 unidade em x_i multiplica os odds por e^{a_i}
$a_i < 0 \Rightarrow OR_i < 1$	Aumento de 1 unidade em x_i reduz os odds por fator e^{a_i}
$a_i = 0 \Rightarrow OR_i = 1$	x_i não tem efeito sobre os odds do evento

Exemplo numérico: se $a_1 = 2,44$, então $OR_1 = e^{2,44} \approx 11,47$. Um aumento de 1 unidade em x_1 eleva os odds do evento em **1047%**, mantidas as demais variáveis constantes.

3. Estimação por Máxima Verossimilhança

3.1 O princípio

A **Máxima Verossimilhança** (MV) encontra os valores de $\mathbf{a}$ e b que **maximizam a probabilidade de se observar exatamente os dados coletados**, dada a estrutura do modelo.

3.2 A função de verossimilhança

Para n observações independentes, com $y_i \in \{0, 1\}$, cada observação segue uma distribuição de Bernoulli com parâmetro $\hat{y}_i$. A verossimilhança conjunta é:

$$\mathcal{L}(\mathbf{a}, b) = \prod_{i=1}^n \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{1-y_i}$$

Cada fator contribui com: - $\hat{y}_i$ quando $y_i = 1$ (probabilidade prevista de sucesso)
- $(1 - \hat{y}_i)$ quando $y_i = 0$ (probabilidade prevista de fracasso)

3.3 A log-verossimilhança

Para evitar *underflow* numérico e transformar o produto em soma, maximiza-se o logaritmo natural:

$$L(\mathbf{a}, b) = \sum_{i=1}^n [y_i \log_e(\hat{y}_i) + (1 - y_i) \log_e(1 - \hat{y}_i)]$$

Propriedades:

- $L \leq 0$ sempre, pois $\log_e(\hat{y}_i) \leq 0$ para $\hat{y}_i \in (0, 1)$
- $L = 0$ somente quando o modelo classifica perfeitamente todos os pontos
- Maximizar L equivale a minimizar a **entropia cruzada binária**, função de perda usada em redes neurais para classificação binária

3.4 Exemplo: preferência por café

Pesquisa com 10 pessoas, das quais 7 responderam “Sim” e 3 “Não”. Parâmetro a estimar: p (proporção que gosta de café).

Verossimilhança (desconsiderando combinatórias — não afetam o maximizador):

$$\mathcal{L}(p) = p^7 \cdot (1 - p)^3$$

Log-verossimilhança:

$$L(p) = 7 \log_e(p) + 3 \log_e(1 - p)$$

Condição de 1ª ordem (ponto crítico):

$$\frac{dL}{dp} = \frac{7}{p} - \frac{3}{1-p} = 0 \implies 7(1-p) = 3p \implies \hat{p} = \frac{7}{10} = 0,7$$

Condição de 2ª ordem (verificação de máximo):

$$\frac{d^2L}{dp^2} = -\frac{7}{p^2} - \frac{3}{(1-p)^2} < 0 \quad \forall p \in (0, 1)$$

Como a segunda derivada é estritamente negativa, $\hat{p} = 0,7$ é um **máximo global** — coincide, como esperado, com a proporção amostral.

```
p_seq <- seq(0.01, 0.99, length.out = 500)
L_cafe <- 7 * log(p_seq) + 3 * log(1 - p_seq)
p_hat <- 7 / 10
L_hat <- 7 * log(p_hat) + 3 * log(1 - p_hat)

mv_df <- data.frame(p = p_seq, L = L_cafe)

ggplot(mv_df, aes(x = p, y = L)) +
  geom_line(color = "#8E44AD", size = 1.3) +
  geom_vline(xintercept = p_hat, linetype = "dashed",
             color = "#C0392B", size = 0.8) +
  geom_hline(yintercept = L_hat, linetype = "dotted",
             color = "gray50", size = 0.7) +
  geom_point(aes(x = p_hat, y = L_hat),
             color = "#C0392B", size = 4, shape = 21,
             fill = "#E74C3C") +
  annotate("text", x = 0.76, y = L_hat - 0.3,
          label = expression(hat(p) == 0.7),
          color = "#C0392B", size = 5, fontface = "bold") +
  annotate("text", x = 0.20, y = L_hat,
          label = paste0("L* = ", round(L_hat, 3)),
          color = "gray40", size = 4, vjust = -0.6) +
  labs(x = "p", y = "L(p)",
       title = "Log-Verossimilhança - Exemplo do Café",
       subtitle = "Máximo em  $\hat{p} = 7/10 = 0,7$ ") +
  theme_minimal(base_size = 13) +
  theme(panel.grid.minor = element_blank())
```

3.5 Log-verossimilhança com preditoras (modelo geral)

Substituindo $\hat{y}_i = \sigma(z_i)$:

$$L(\mathbf{a}, b) = \sum_{i=1}^n \left[y_i \log_e \left(\frac{1}{1 + e^{-z_i}} \right) + (1 - y_i) \log_e \left(\frac{e^{-z_i}}{1 + e^{-z_i}} \right) \right]$$

Não existe solução analítica fechada. Os coeficientes são estimados por algoritmos de otimização iterativa:

Algoritmo	Descrição	Implementação
Newton-Raphson (IRLS)	Usa gradiente e hessiana exata; converge rapidamente	<code>glm()</code> no R
BFGS / L-BFGS	Aproxima a hessiana; eficiente para muitas variáveis	<code>statsmodels</code> , <code>sklearn</code>
Gradiente Descendente	Atualização por mini-batch; escalável a Big Data	Redes neurais, TensorFlow

4. Avaliação do Modelo

4.1 Pseudo- R^2 de McFadden

Na regressão logística não existe um R^2 com interpretação geométrica direta. O **pseudo- R^2 de McFadden** (1974) é a medida mais usada:

$$R_{\text{McFadden}}^2 = 1 - \frac{L^*}{L_0} = 1 - \frac{L^*}{N_1 \log_e N_1 + N_0 \log_e N_0 - (N_1 + N_0) \log_e (N_1 + N_0)}$$

onde:

Símbolo	Definição
L^*	Log-verossimilhança máxima do modelo ajustado (≤ 0)
L_0	Log-verossimilhança do modelo nulo (só intercepto)
N_1	Número de sucessos ($y = 1$)
N_0	Número de fracassos ($y = 0$)

O denominador L_0 equivale à log-verossimilhança de um modelo que prevê $\hat{p} = N_1/N$ para todos — o “pior” modelo útil.

Tabela de interpretação:

Valor de R_{McFadden}^2	Avaliação
$< 0,20$	Ajuste fraco
$0,20 - 0,40$	Bom ajuste
$> 0,40$	Ajuste muito bom

Na regressão logística, valores relativamente baixos de pseudo- R^2 são esperados e não indicam necessariamente um modelo ruim. Um $R_{\text{McFadden}}^2 \approx 0,4$ em regressão logística é comparável a um $R^2 \approx 0,9$ em regressão linear.

4.1 R^2 de McFadden – Fórmula detalhada

4.1.1 Fórmula correta (conceitual)

$$R_{\text{McFadden}}^2 = 1 - \frac{\ln L^*}{\ln L_0}$$

- $\ln L^*$ = log-verossimilhança do **modelo completo** (com preditores).
- $\ln L_0$ = log-verossimilhança do **modelo nulo** (apenas intercepto).

Ambos são números **negativos** (ou zero).

Como o modelo completo é pelo menos tão bom quanto o nulo,

$$\ln L^* \geq \ln L_0$$

(menos negativo).

Portanto

$$0 \leq \frac{\ln L^*}{\ln L_0} \leq 1$$

e o R^2 fica entre 0 e 1.

O erro comum é omitir o termo $1 - \dots$, escrevendo apenas

$$-\frac{L^*}{L_0}$$

, o que gera valor e sinal incorretos.

4.1.2 Fórmula detalhada do modelo nulo (L_0)

Em regressão logística binária ($Y \in \{0, 1\}$), o modelo nulo prevê a mesma probabilidade para todos: $\hat{p} = \frac{N_1}{N}$, onde:

- N_1 = observações com $Y = 1$
- N_0 = observações com $Y = 0$
- $N = N_1 + N_0$

A log-verossimilhança do modelo nulo é:

$$\ln L_0 = N_1 \ln\left(\frac{N_1}{N}\right) + N_0 \ln\left(\frac{N_0}{N}\right)$$

Expandindo os logaritmos:

$$\ln L_0 = N_1 \ln N_1 + N_0 \ln N_0 - N \ln N$$

Essa expressão aparecia no denominador da caixa original e está correta — o erro estava apenas na fórmula geral do R^2 .

4.1.3 Fórmula completa para uso prático

$$R_{\text{McFadden}}^2 = 1 - \frac{\ln L^*}{N_1 \ln N_1 + N_0 \ln N_0 - N \ln N}$$

Ou, com a notação de logaritmo natural ($\log_e$):

$$R_{\text{McFadden}}^2 = 1 - \frac{L^*}{N_1 \log_e N_1 + N_0 \log_e N_0 - (N_1 + N_0) \log_e (N_1 + N_0)}$$

4.1.4 Exemplo numérico

Dados: $N_1 = 60$, $N_0 = 40$, $N = 100$.

Modelo nulo:

$$\ln L_0 = 60 \ln(0,6) + 40 \ln(0,4) \approx -67,3$$

Suponha que o modelo completo tenha $\ln L^* = -50,0$:

$$R_{\text{McFadden}}^2 = 1 - \frac{-50,0}{-67,3} \approx 1 - 0,743 = 0,257$$

Interpretação: o modelo completo reduz cerca de **25,7%** da incerteza (deviance) em relação ao modelo nulo — análogo ao R^2 tradicional, mas na escala da log-verossimilhança.

4.2 Acurácia e taxa de erro aparente

$$\text{Acurácia} = \frac{\text{classificações corretas}}{n}, \quad \text{Taxa de Erro} = 1 - \text{Acurácia}$$

A **taxa de erro aparente** é calculada no conjunto de treino e tende a subestimar o erro real. O correto é calculá-la no **conjunto de teste** (dados não vistos durante o ajuste).

4.3 Matriz de confusão

Para um limiar de decisão τ (geralmente 0,5), classifica-se:

$$\hat{y}_i = \begin{cases} 1 & \text{se } \hat{p}_i \geq \tau \\ 0 & \text{se } \hat{p}_i < \tau \end{cases}$$

A **matriz de confusão** cruza os valores reais com as previsões:

	Predito 0	Predito 1
Real 0	Verdadeiro Negativo (VN)	Falso Positivo (FP) — Erro Tipo I
Real 1	Falso Negativo (FN) — Erro Tipo II	Verdadeiro Positivo (VP)

Métricas derivadas:

$$\text{Sensibilidade} = \frac{VP}{VP + FN} \quad \text{Especificidade} = \frac{VN}{VN + FP}$$

$$\text{Precisão} = \frac{VP}{VP + FP} \quad \text{F1-Score} = 2 \cdot \frac{\text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}}$$

Escolha do limiar τ : em contextos onde falsos negativos são muito custosos (ex.: diagnóstico de câncer), adota-se $\tau < 0,5$ para aumentar a sensibilidade, aceitando mais falsos positivos. A curva ROC auxilia nessa escolha.

4.4 Curva ROC e AUC

A **Curva ROC** (*Receiver Operating Characteristic*) plota a **Sensibilidade (TPR)** versus **(1 – Especificidade) = FPR** para **todos os possíveis limiares $\tau \in [0, 1]$** .

A **AUC** (*Area Under the ROC Curve*) resume o desempenho:

AUC	Interpretação
0,5	Classificador aleatório (sem poder preditivo)
0,70 – 0,80	Desempenho aceitável
0,80 – 0,90	Bom desempenho
> 0,90	Desempenho excelente

A AUC também pode ser interpretada como a probabilidade de que, dado um par aleatório (um positivo e um negativo), o modelo atribua maior probabilidade ao positivo: $\text{AUC} = P(\hat{p}_1 > \hat{p}_0)$.

5. Testes de Hipóteses

5.1 Teste da razão de verossimilhanças (TRV) — modelo global

Avalia se o modelo ajustado é **globalmente** significativo em relação ao modelo nulo (sem preditoras).

Hipóteses:

$$H_0 : a_1 = a_2 = \dots = a_p = 0 \quad \text{vs.} \quad H_1 : \exists i : a_i \neq 0$$

Estatística de teste:

$$G = 2(L^* - L_0) \xrightarrow{H_0} \chi^2_{(p)}$$

onde p é o número de preditoras (graus de liberdade). Rejeita-se H_0 se o valor-p $< \alpha$.

Protocolo de decisão:

1. Definir a população de interesse
2. Enunciar H_0 e H_1
3. Escolher o TRV como teste
4. Fixar $\alpha = 0,05$
5. Calcular $G = 2(L^* - L_0)$
6. Obter valor-p: $P(\chi^2_{(p)} \geq G)$
7. Decisão: rejeitar H_0 se valor-p $< 0,05$; caso contrário, não rejeitar

5.2 Teste de Wald — coeficientes individuais

Avalia a significância de **cada coeficiente** separadamente.

Hipóteses para o i -ésimo coeficiente:

$$H_0 : a_i = 0 \quad \text{vs.} \quad H_1 : a_i \neq 0$$

Estatística de Wald:

$$W_i = \left(\frac{\hat{a}_i}{SE(\hat{a}_i)} \right)^2 \xrightarrow{H_0} \chi^2_{(1)}$$

O erro padrão $SE(\hat{a}_i)$ é a raiz quadrada do i -ésimo elemento diagonal da matriz de covariância dos estimadores:

$$\widehat{\text{Var}}(\hat{\mathbf{a}}) = (\mathbf{X}^\top \widehat{W} \mathbf{X})^{-1}, \quad \widehat{W} = \text{diag}\{\hat{y}_i(1 - \hat{y}_i)\}$$

A estatística equivalente em escala normal é $z_i = \hat{a}_i / SE(\hat{a}_i) \sim \mathcal{N}(0, 1)$, reportada como “z value” na saída do R.

Protocolo de decisão:

1. Definir a população
2. $H_0 : a_i = 0$ e $H_1 : a_i \neq 0$
3. Escolher o Teste de Wald
4. Fixar $\alpha = 0,05$
5. Calcular $W_i = \hat{a}_i^2 / SE(\hat{a}_i)^2$
6. Obter valor-p: $P(\chi^2_{(1)} \geq W_i)$
7. Rejeitar H_0 se valor-p $< 0,05$

6. Previsão com o Modelo

Com o modelo validado, a probabilidade predita para uma nova observação $\mathbf{x}^* = (x_1^*, \dots, x_p^*)$ é:

$$\hat{p} = \frac{1}{1 + e^{-(\hat{a}_1 x_1^* + \dots + \hat{a}_p x_p^* + \hat{b})}}$$

A classificação usa o limiar τ :

$$\hat{y} = \begin{cases} 1 & \text{se } \hat{p} \geq \tau \\ 0 & \text{se } \hat{p} < \tau \end{cases}$$

O limiar padrão $\tau = 0,5$ pode ser ajustado com base na curva ROC para equilibrar sensibilidade e especificidade conforme o contexto.

7. Implementação em R

Como usar: ajuste as **6 variáveis** do bloco CONFIGURAÇÃO abaixo para o seu arquivo `.csv`. Todo o restante do código funciona automaticamente. Se o arquivo não existir, dados simulados são gerados para demonstração.

7.1 Pacotes

```
# Instalação automática dos pacotes ausentes
pkgs_necessarios <- c("ggplot2", "dplyr", "pROC", "caret",
                      "DescTools", "gridExtra", "knitr")
pkgs_ausentes    <- pkgs_necessarios[
  !pkgs_necessarios %in% installed.packages()[, "Package"]
]
if (length(pkgs_ausentes) > 0)
  install.packages(pkgs_ausentes, repos = "https://cran.r-project.org",
                  quiet = TRUE)

suppressPackageStartupMessages({
  library(ggplot2)
  library(dplyr)
  library(pROC)
  library(caret)
  library(DescTools)
  library(gridExtra)
  library(knitr)
})
```

7.2 Configuração e importação dos dados

```
#
# CONFIGURAÇÃO - ajuste apenas estas variáveis
#
CAMINHO_CSV      <- "dados.csv"      # caminho para o arquivo .csv
VARIABEL_RESPOSTA <- "y"            # coluna resposta (valores 0 e 1)
VARIABLES_PRED   <- c("x1", "x2")   # vetor com nomes das preditoras
LIMiar_DECISAO    <- 0.5             # limiar de classificação
PROPORCAO_TREINO  <- 0.70           # fração para treino
SEMENTE          <- 42              # semente para reprodutibilidade
#

# Geração de dados simulados (apenas se o .csv não existir)
if (!file.exists(CAMINHO_CSV)) {
  message("Arquivo não encontrado - gerando dados simulados.")
  set.seed(SEMENTE)
  n_sim <- 350
  x1_sim <- round(runif(n_sim, 18, 65), 1) # ex.: idade
  x2_sim <- round(rnorm(n_sim, 50, 10), 1)  # ex.: pontuação
  z_sim <- 0.07 * x1_sim + 0.06 * x2_sim - 7.0
  p_sim <- 1 / (1 + exp(-z_sim))
}
```

```

y_sim <- rbinom(n_sim, 1, p_sim)
df_sim <- data.frame(y = y_sim, x1 = x1_sim, x2 = x2_sim)
write.csv(df_sim, CAMINHO_CSV, row.names = FALSE)
message(sprintf("Arquivo '%s' criado com %d observações.", CAMINHO_CSV, n_sim))
}

# Leitura
raw <- read.csv(CAMINHO_CSV, stringsAsFactors = FALSE)
cat(sprintf("Dataset lido: %d linhas × %d colunas\n", nrow(raw), ncol(raw)))
cat("Colunas:", paste(names(raw), collapse = ", "), "\n")

# Validação das colunas configuradas
cols_req <- c(VARIAVEL_RESPOSTA, VARIAVEIS_PRED)
cols_missing <- setdiff(cols_req, names(raw))
if (length(cols_missing) > 0)
  stop("Colunas ausentes no CSV: ", paste(cols_missing, collapse = ", "),
       "\nAjuste VARIAVEL_RESPOSTA e VARIAVEIS_PRED.")

# Data frame de trabalho
dados <- raw[, cols_req]
names(dados)[1] <- "y"
dados$y <- as.integer(as.character(dados$y))

# Remoção de NAs
n_antes <- nrow(dados)
dados <- na.omit(dados)
n_depois <- nrow(dados)
if (n_antes > n_depois)
  cat(sprintf("Removidas %d linha(s) com NA.\n", n_antes - n_depois))

cat(sprintf("Observações válidas: %d\n", n_depois))

```

7.3 Análise exploratória

```

N1 <- sum(dados$y == 1)
N0 <- sum(dados$y == 0)
N <- nrow(dados)

cat(sprintf(
  "Classe 1 (sucesso): %d (%.1f%%)\nClasse 0 (fracasso): %d (%.1f%%)\n",
  N1, 100 * N1 / N, N0, 100 * N0 / N
))

dados$y_fator <- factor(dados$y, levels = c(0, 1),
  labels = c("Fracasso (0)", "Sucesso (1)"))
pal <- c("Fracasso (0)" = "#C0392B", "Sucesso (1)" = "#27AE60")

# Gráfico de dispersão
p_disp <- ggplot(dados,
  aes_string(x = VARIAVEIS_PRED[1],
    y = if (length(VARIAVEIS_PRED) > 1) VARIAVEIS_PRED[2]

```

```

        else VARIAVEIS_PRED[1],
              color = "y_fator", shape = "y_fator")) +
geom_jitter(alpha = 0.65, size = 1.8, width = 0.25, height = 0.25) +
scale_color_manual(values = pal) +
scale_shape_manual(values = c("Fracasso (0)" = 16, "Sucesso (1)" = 17)) +
labs(title = "Dispersão por Classe",
      color = NULL, shape = NULL) +
theme_minimal(base_size = 11) +
theme(legend.position = "bottom")

# Boxplots por preditora
plots_bp <- lapply(VARIAVEIS_PRED, function(v) {
  ggplot(dados, aes_string(x = "y_fator", y = v, fill = "y_fator")) +
    geom_boxplot(alpha = 0.75, outlier.shape = 21,
                 outlier.fill = "white", outlier.stroke = 0.5,
                 outlier.size = 1.5) +
    scale_fill_manual(values = pal) +
    labs(x = NULL, y = v,
         title = paste("Distribuição de", v)) +
    theme_minimal(base_size = 11) +
    theme(legend.position = "none",
          axis.text.x = element_text(size = 9))
})

do.call(grid.arrange, c(list(p_disp), plots_bp, list(ncol = 3)))

```

7.4 Divisão treino/teste e ajuste do modelo

```

set.seed(SEMENTE)

# Partição estratificada (mantém proporção de classes)
idx_treino <- createDataPartition(dados$y, p = PROPORCAO_TREINO, list = FALSE)
treino     <- dados[idx_treino, ]
teste     <- dados[-idx_treino, ]

cat(sprintf("Treino: %d obs. | Teste: %d obs.\n", nrow(treino), nrow(teste)))

# Fórmula do modelo
formula_modelo <- as.formula(
  paste("y ~", paste(VARIAVEIS_PRED, collapse = " + "))
)

# Ajuste via GLM - família binomial, ligação logit (MV via IRLS)
modelo <- glm(formula_modelo, data = treino,
              family = binomial(link = "logit"))

print(summary(modelo))

```

7.5 Coeficientes e odds ratios

```
ci_logit <- suppressMessages(confit(modelo))
or_table <- exp(cbind(OR = coef(modelo), ci_logit))
colnames(or_table) <- c("OR", "IC 2,5%", "IC 97,5%")

cat("=== Odds Ratios com Intervalo de Confiança 95% ===\n")
kable(round(or_table, 4),
      caption = "Tabela 1. Coeficientes, Odds Ratios e IC 95%")
```

7.6 Pseudo-R² de McFadden

```
L_star <- as.numeric(logLik(modelo))
L_nulo_val <- N1 * log(N1) + N0 * log(N0) - N * log(N)
r2_mcf <- -L_star / L_nulo_val

cat(sprintf("Log-verossimilhança do modelo (L*): %.4f\n", L_star))
cat(sprintf("Log-verossimilhança do nulo (L0): %.4f\n", L_nulo_val))
cat(sprintf("Pseudo-R2 de McFadden: %.4f\n", r2_mcf))
cat("Avaliação:",
    dplyr::case_when(
      r2_mcf < 0.20 ~ "Ajuste fraco (< 0,20)",
      r2_mcf < 0.40 ~ "Bom ajuste (0,20 - 0,40)",
      TRUE ~ "Ajuste muito bom (> 0,40)"
    ), "\n")
```

7.7 Teste da razão de verossimilhanças

```
modelo_nulo <- glm(y ~ 1, data = treino, family = binomial(link = "logit"))

G <- 2 * (as.numeric(logLik(modelo)) - as.numeric(logLik(modelo_nulo)))
gl_trv <- length(VARIAVEIS_PRED)
pval_trv <- 1 - pchisq(G, df = gl_trv)

cat("H0: a1 = a2 = ... = ap = 0\n")
cat("H1: pelo menos um ai > 0\n\n")
cat(sprintf("Estatística G: %.4f\n", G))
cat(sprintf("Graus de liberdade: %d\n", gl_trv))
cat(sprintf("Valor-p: %.2e\n", pval_trv))
cat("Decisão (α = 0,05):",
    ifelse(pval_trv < 0.05,
           "Rejeitar H0 - modelo globalmente significativo.",
           "Não rejeitar H0."), "\n")
```

7.8 Teste de Wald

```

coefs_v <- coef(modelo)
ep_v <- sqrt(diag(vcov(modelo)))
w_stat_v <- (coefs_v / ep_v)^2
p_wald_v <- 1 - pchisq(w_stat_v, df = 1)

wald_df <- data.frame(
  Coeficiente = round(coefs_v, 4),
  Erro_Padiao = round(ep_v, 4),
  Estatistica_W = round(w_stat_v, 4),
  Valor_p = round(p_wald_v, 6),
  Decisao = ifelse(p_wald_v < 0.05, "Rejeitar H0 (*)", "Não rejeitar H0")
)

cat("H0: a_i = 0 | H1: a_i ≠ 0\n\n")
kable(wald_df, caption = "Tabela 2. Teste de Wald para cada coeficiente")

```

7.9 Previsão e avaliação no conjunto de teste

```

prob_teste <- predict(modelo, newdata = teste, type = "response")
y_pred_r <- ifelse(prob_teste >= LIMiar_DECISAO, 1, 0)

acuracia_r <- mean(y_pred_r == teste$y)
taxa_erro_r <- 1 - acuracia_r

cat(sprintf("Limiar de decisão ( ): %.2f\n", LIMiar_DECISAO))
cat(sprintf("Acurácia: %.4f (%.1f%%)\n",
  acuracia_r, 100 * acuracia_r))
cat(sprintf("Taxa de erro: %.4f (%.1f%%)\n\n",
  taxa_erro_r, 100 * taxa_erro_r))

cm_r <- confusionMatrix(
  factor(y_pred_r, levels = c(0, 1)),
  factor(teste$y, levels = c(0, 1)),
  positive = "1"
)
print(cm_r)

```

7.10 Visualizações: matriz de confusão e curva ROC

```

# Matriz de confusão
cm_df_r <- as.data.frame(cm_r$table)
names(cm_df_r) <- c("Predito", "Real", "N")

p_mc <- ggplot(cm_df_r, aes(x = Real, y = Predito, fill = N)) +
  geom_tile(color = "white", size = 1.2) +
  geom_text(aes(label = N), size = 9, fontface = "bold", color = "white") +
  scale_fill_gradient(low = "#AED6F1", high = "#1A5276") +
  scale_x_discrete(labels = c("0" = "Real 0", "1" = "Real 1")) +
  scale_y_discrete(labels = c("0" = "Pred 0", "1" = "Pred 1")) +

```

```

labs(title = "Matriz de Confusão",
      x = "Valor Real", y = "Valor Predito", fill = "n") +
theme_minimal(base_size = 12) +
theme(panel.grid = element_blank(),
      axis.text = element_text(size = 11))

# Curva ROC
roc_r <- roc(teste$y, prob_teste, quiet = TRUE)
auc_r <- auc(roc_r)
roc_df_r <- data.frame(
  FPR = 1 - roc_r$specificities,
  TPR = roc_r$sensitivities
)

p_roc <- ggplot(roc_df_r, aes(x = FPR, y = TPR)) +
  geom_ribbon(aes(ymin = 0, ymax = TPR), fill = "#2980B9", alpha = 0.15) +
  geom_line(color = "#2980B9", size = 1.3) +
  geom_abline(slope = 1, intercept = 0,
              linetype = "dashed", color = "gray50") +
  annotate("text", x = 0.60, y = 0.12,
           label = sprintf("AUC = %.3f", auc_r),
           size = 5, color = "#2980B9", fontface = "bold") +
  labs(title = "Curva ROC",
       x = "1 - Especificidade (FPR)",
       y = "Sensibilidade (TPR)") +
  coord_equal() +
  theme_minimal(base_size = 12)

grid.arrange(p_mc, p_roc, ncol = 2)

cat(sprintf("\nAUC: %.4f - %s\n", auc_r,
  dplyr::case_when(
    auc_r >= 0.90 ~ "Excelente",
    auc_r >= 0.80 ~ "Bom",
    auc_r >= 0.70 ~ "Aceitável",
    TRUE ~ "Fraco - revisar o modelo"
  )))

```

7.11 Previsão para nova observação

```

# Defina aqui a nova observação (um valor por preditora)
valores_novos_r <- c(45, 55) # altere conforme sua análise
#

nova_obs_r <- setNames(
  as.data.frame(t(valores_novos_r)),
  VARIAVEIS_PRED
)

prob_nova_r <- predict(modelo, newdata = nova_obs_r, type = "response")
classe_nova_r <- ifelse(prob_nova_r >= LIMIAR_DECISAO,

```



```

        "Sucesso (1)", "Fracasso (0)")

for (v in VARIAVEIS_PRED)
  cat(sprintf("%s = %.1f\n", v, nova_obs_r[[v]]))
cat(sprintf("\nProbabilidade estimada : %.4f\n", prob_nova_r))
cat(sprintf("Classificação ( = %.2f): %s\n", LIMIAR_DECISAO, classe_nova_r))

```

8. Implementação em Python

Como usar: ajuste as **6 variáveis** do bloco **# CONFIGURAÇÃO** abaixo. O código usa **statsmodels** para inferência completa e **scikit-learn** para métricas de classificação.

8.1 Dependências

```

# Instale com: pip install numpy pandas matplotlib seaborn scikit-learn statsmodels scipy
import os, sys, warnings
import numpy          as np
import pandas         as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn        as sns
from scipy import stats
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix, classification_report,
    roc_curve, auc, ConfusionMatrixDisplay, accuracy_score
)
import statsmodels.api as sm

warnings.filterwarnings("ignore")
plt.rcParams.update({"figure.dpi": 130, "font.size": 11})

```

8.2 Configuração e importação dos dados

```

#
# CONFIGURAÇÃO - ajuste apenas estas variáveis
#
CAMINHO_CSV      = "dados.csv"
VARIABEL_RESPOSTA = "y"
VARIAVEIS_PRED   = ["x1", "x2"]
LIMIAR_DECISAO   = 0.5
PROPORCAO_TESTE  = 0.30
SEMENTE          = 42
#

```

```

# Dados simulados (se o .csv não existir)
if not os.path.exists(CAMINHO_CSV):
    print(f"Arquivo não encontrado - gerando dados simulados.")
    rng = np.random.default_rng(SEMENTE)
    n_s = 350
    x1_s = rng.uniform(18, 65, n_s).round(1)
    x2_s = rng.normal(50, 10, n_s).round(1)
    z_s = 0.07 * x1_s + 0.06 * x2_s - 7.0
    p_s = 1 / (1 + np.exp(-z_s))
    y_s = rng.binomial(1, p_s, n_s)
    pd.DataFrame({"y": y_s, "x1": x1_s, "x2": x2_s}).to_csv(CAMINHO_CSV, index=False)
    print(f"{CAMINHO_CSV} criado com {n_s} observações.")

# Leitura
raw = pd.read_csv(CAMINHO_CSV)
print(f"Dataset: {raw.shape[0]} linhas x {raw.shape[1]} colunas")
print(f"Colunas: {'', '.join(raw.columns)}")

# Validação
cols_faltando = [c for c in [VARIAVEL_RESPOSTA] + VARIAVEIS_PRED
                  if c not in raw.columns]
if cols_faltando:
    sys.exit(f"Colunas ausentes: {'', '.join(cols_faltando)}\n"
            "Ajuste VARIAVEL_RESPOSTA e VARIAVEIS_PRED.")

dados = raw[[VARIAVEL_RESPOSTA] + VARIAVEIS_PRED].copy()
dados.rename(columns={VARIAVEL_RESPOSTA: "y"}, inplace=True)
dados["y"] = dados["y"].astype(int)
n_antes = len(dados)
dados.dropna(inplace=True)
if len(dados) < n_antes:
    print(f"Removidas {n_antes - len(dados)} linha(s) com NA.")

N1, N0, N = dados["y"].sum(), (dados["y"] == 0).sum(), len(dados)
print(f"Classe 1: {N1} ({100*N1/N:.1f}%) Classe 0: {N0} ({100*N0/N:.1f}%)")

```

8.3 Análise exploratória

```

fig = plt.figure(figsize=(13, 4))
gs = gridspec.GridSpec(1, 1 + len(VARIAVEIS_PRED), figure=fig)

# Dispersão
ax0 = fig.add_subplot(gs[0, 0])
cores = {0: "#C0392B", 1: "#27AE60"}
for cls, grp in dados.groupby("y"):
    ax0.scatter(grp[VARIAVEIS_PRED[0]],
                grp[VARIAVEIS_PRED[1]] if len(VARIAVEIS_PRED) > 1 else grp[VARIAVEIS_PRED[0]],
                c=cores[cls],
                label="Sucesso (1)" if cls == 1 else "Fracasso (0)",
                alpha=0.65, edgecolors="white", s=45)
ax0.set_xlabel(VARIAVEIS_PRED[0]); ax0.set_title("Dispersão por Classe")

```

```

ax0.set_ylabel(VARIAVEIS_PRED[1] if len(VARIAVEIS_PRED) > 1 else "")
ax0.legend(fontsize=9)

# Boxplots
for k, var in enumerate(VARIAVEIS_PRED):
    ax = fig.add_subplot(gs[0, k + 1])
    grupos = [dados.loc[dados["y"]==0, var].values,
              dados.loc[dados["y"]==1, var].values]
    bp = ax.boxplot(grupos, patch_artist=True, widths=0.5,
                    medianprops=dict(color="black", linewidth=2))
    for b, cor in zip(bp["boxes"], ["#C0392B", "#27AE60"]):
        b.set_facecolor(cor); b.set_alpha(0.75)
    ax.set_xticklabels(["Fracasso (0)", "Sucesso (1)"], fontsize=9)
    ax.set_title(f"Distribuição de {var}"); ax.set_ylabel(var)

plt.suptitle("Análise Exploratória", fontweight="bold", y=1.01)
plt.tight_layout()
plt.savefig("py_eda.png", bbox_inches="tight"); plt.show()

```

8.4 Divisão treino/teste e ajuste do modelo

```

X = dados[VARIAVEIS_PRED].values
y = dados["y"].values

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=PROPORCAO_TESTE,
    random_state=SEMENTE, stratify=y
)
print(f"Treino: {len(y_tr)} obs. | Teste: {len(y_te)} obs.")

# statsmodels: inferência estatística completa
X_tr_sm = sm.add_constant(X_tr)
X_te_sm = sm.add_constant(X_te)
modelo_sm = sm.Logit(y_tr, X_tr_sm).fit(dispatch=False)
print(modelo_sm.summary())

```

8.5 Coeficientes e odds ratios

```

params = modelo_sm.params
ep = modelo_sm.bse
ci = modelo_sm.conf_int()

nomes = ["Intercepto"] + VARIAVEIS_PRED
df_coef = pd.DataFrame({
    "Coeficiente" : params.values,
    "Erro Padrão" : ep.values,
    "OR" : np.exp(params.values),
    "IC 2,5% (OR)": np.exp(ci.iloc[:, 0].values),
    "IC 97,5% (OR)": np.exp(ci.iloc[:, 1].values),
})

```

```

    "Valor-p"      : modelo_sm.pvalues.values
}, index=nomes)

print(df_coef.round(4).to_string())

```

8.6 Pseudo-R² de McFadden

```

L_star_py = modelo_sm.llf
N1_tr     = int(y_tr.sum())
N0_tr     = len(y_tr) - N1_tr
N_tr      = len(y_tr)
L0_py     = N1_tr*np.log(N1_tr) + N0_tr*np.log(N0_tr) - N_tr*np.log(N_tr)

r2_mcf_py = -L_star_py / L0_py
print(f"L* = {L_star_py:.4f}    L0 = {L0_py:.4f}")
print(f"Pseudo-R2 de McFadden: {r2_mcf_py:.4f}")
print(f"(statsmodels nativo):    {modelo_sm.prsquared:.4f}")

avaliacao = ("Fraco"      if r2_mcf_py < 0.20 else
             "Bom"       if r2_mcf_py < 0.40 else
             "Muito bom")
print(f"Avaliação: {avaliacao}")

```

8.7 Teste da razão de verossimilhanças

```

modelo_nulo_py = sm.Logit(y_tr, np.ones(N_tr)).fit(disps=False)
G_py          = 2 * (modelo_sm.llf - modelo_nulo_py.llf)
gl_py         = len(VARIAVEIS_PRED)
pval_py       = 1 - stats.chi2.cdf(G_py, df=gl_py)

print(f"H0: todos os coeficientes = 0")
print(f"Estatística G:      {G_py:.4f}")
print(f"Graus de liberdade: {gl_py}")
print(f"Valor-p:              {pval_py:.2e}")
print("Decisão:", "Rejeitar H0." if pval_py < 0.05 else "Não rejeitar H0.")

```

8.8 Teste de Wald

```

w_py = (params / ep) ** 2
pw_py = 1 - stats.chi2.cdf(w_py, df=1)

df_wald = pd.DataFrame({
    "Coeficiente" : params.values.round(4),
    "Erro Padrão"  : ep.values.round(4),
    "Estatística W": w_py.values.round(4),
    "Valor-p"      : pw_py.values.round(6),
    "Decisão"      : ["Rejeitar H0 (*)" if p < 0.05 else "Não rejeitar H0"]
})

```

```

        for p in pw_py]
}, index=nomes)

print("H0: a_i = 0 | H1: a_i > 0\n")
print(df_wald.to_string())

```

8.9 Previsão e avaliação no conjunto de teste

```

# scikit-learn para previsão e métricas de classificação
modelo_sk = LogisticRegression(max_iter=1000, random_state=SEMENTE)
modelo_sk.fit(X_tr, y_tr)

prob_te = modelo_sk.predict_proba(X_te)[:, 1]
y_pred_py = (prob_te >= LIMIAR_DECISAO).astype(int)

acc_py = accuracy_score(y_te, y_pred_py)
err_py = 1 - acc_py
print(f"Acurácia: {acc_py:.4f} ({100*acc_py:.1f}%)")
print(f"Taxa de erro: {err_py:.4f} ({100*err_py:.1f}%)")
print(classification_report(y_te, y_pred_py,
                             target_names=["Fracasso (0)", "Sucesso (1)"]))

```

8.10 Visualizações: matriz de confusão e curva ROC

```

cm_py = confusion_matrix(y_te, y_pred_py)
VN, FP, FN, VP = cm_py.ravel()
sens = VP / (VP + FN) if (VP + FN) > 0 else 0
spec = VN / (VN + FP) if (VN + FP) > 0 else 0
prec = VP / (VP + FP) if (VP + FP) > 0 else 0
f1 = 2*prec*sens / (prec+sens) if (prec+sens) > 0 else 0

print(f"Sensibilidade: {sens:.4f} Especificidade: {spec:.4f}")
print(f"Precisão: {prec:.4f} F1-Score: {f1:.4f}")

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

# Matriz de confusão
ConfusionMatrixDisplay(cm_py,
                        display_labels=["Fracasso (0)", "Sucesso (1)"]).plot(
    ax=axes[0], colorbar=False, cmap="Blues")
axes[0].set_title(f"Matriz de Confusão\nAcurácia = {acc_py:.3f}")

# Curva ROC
fpr_py, tpr_py, _ = roc_curve(y_te, prob_te)
auc_py = auc(fpr_py, tpr_py)
axes[1].fill_between(fpr_py, tpr_py, alpha=0.15, color="#2980B9")
axes[1].plot(fpr_py, tpr_py, color="#2980B9", lw=2,
             label=f"AUC = {auc_py:.3f}")
axes[1].plot([0,1],[0,1], "k--", lw=1, label="Aleatório")

```

```

axes[1].set_xlabel("1 - Especificidade"); axes[1].set_ylabel("Sensibilidade")
axes[1].set_title("Curva ROC"); axes[1].legend(loc="lower right")
axes[1].set_aspect("equal")

plt.tight_layout()
plt.savefig("py_mc_roc.png", bbox_inches="tight"); plt.show()
print(f"\nAUC: {auc_py:.4f}")

```

8.11 Previsão para nova observação

```

# Defina aqui a nova observação (um valor por preditora)
valores_novos_py = [45, 55]
#

nova_obs_py = np.array(valores_novos_py).reshape(1, -1)
prob_nova_py = modelo_sk.predict_proba(nova_obs_py)[0, 1]
classe_nova = "Sucesso (1)" if prob_nova_py >= LIMIAR_DECISAO else "Fracasso (0)"

for var, val in zip(VARIAVEIS_PRED, valores_novos_py):
    print(f"{var} = {val}")
print(f"\nProbabilidade estimada: {prob_nova_py:.4f}")
print(f"Classificação ( = {LIMIAR_DECISAO}): {classe_nova}")

```

9. Comparativo R vs Python

Etapas	R	Python
Ajuste do modelo	<code>glm(..., family = binomial)</code>	<code>sm.Logit().fit()</code>
Coefficientes	<code>coef(modelo)</code>	<code>modelo.params</code>
Intervalos de confiança	<code>confint(modelo)</code>	<code>modelo.conf_int()</code>
Odds Ratios	<code>exp(coef(modelo))</code>	<code>np.exp(modelo.params)</code>
Pseudo-R ²	<code>DescTools::PseudoR2()</code>	<code>modelo.prsquared</code>
TRV global	<code>anova(nulo, modelo, test="Chisq")</code>	<code>2*(modelo.llf - nulo.llf) → chi2.cdf</code>
Teste de Wald	<code>summary(modelo) (z value)</code>	<code>modelo.tvalues²</code>
Probabilidade predita	<code>predict(modelo, type="response")</code>	<code>modelo.predict(X_sm)</code>
Matriz de confusão	<code>caret::confusionMatrix()</code>	<code>sklearn.metrics.confusion_matrix</code>
Curva ROC	<code>pROC::roc() + auc()</code>	<code>sklearn.metrics.roc_curve + auc()</code>

10. Checklist da Análise Completa

- ☒ Importação e validação do arquivo .csv

- ☒ Análise exploratória: dispersão e boxplots por classe
- ☒ Balanceamento de classes verificado
- ☒ Divisão estratificada treino/teste
- ☒ Ajuste do modelo logístico por Máxima Verossimilhança
- ☒ Coeficientes, erros-padrão e odds ratios com IC 95%
- ☒ Pseudo- R^2 de McFadden calculado e interpretado
- ☒ Teste da Razão de Verossimilhanças (significância global)
- ☒ Teste de Wald (significância individual de cada coeficiente)
- ☒ Acurácia e taxa de erro no conjunto de teste
- ☒ Matriz de confusão com sensibilidade, especificidade, F1
- ☒ Curva ROC e AUC calculadas e interpretadas
- ☒ Previsão para nova observação realizada

Tutorial produzido com base nas notas de aula de análise de regressão logística binária, revisadas, formalizadas e expandidas com conteúdos complementares e implementações completas em R e Python.